Career PathFinder — Solution Documentation

HCL Amplified National Hackathon — AI-Powered Personalized Learning Path Recommender

1. Executive Summary

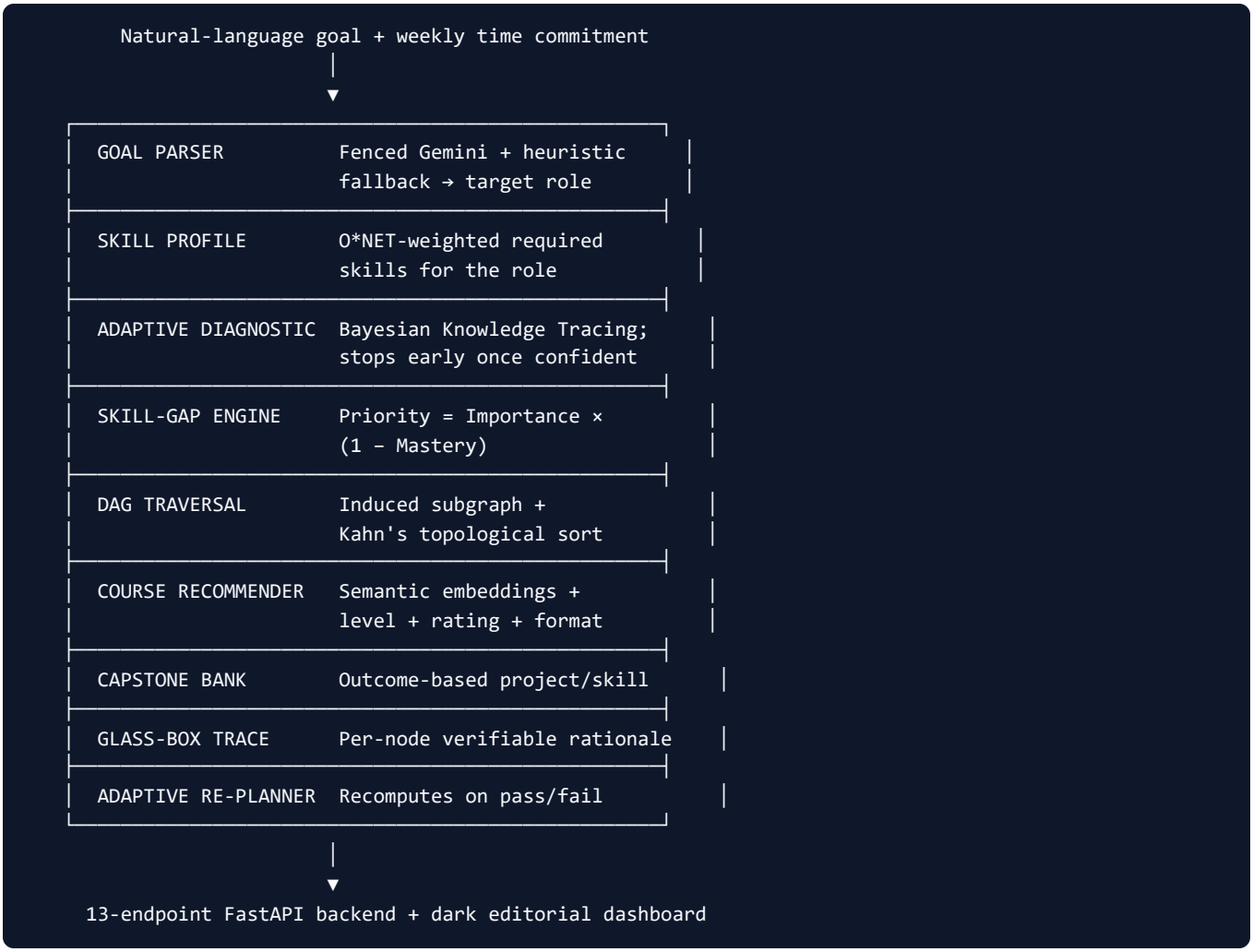
Career PathFinder is a **grounded, adaptive, and explainable** learning-path engine. Given a career goal in plain English, it measures the learner's actual competency, computes the labour-market skill gap, and sequences a prerequisite-correct roadmap of real courses — then shows its reasoning for every step and re-plans dynamically as the learner progresses or struggles.

The guiding design principle is a hard separation of concerns:

> **The deterministic engine decides *what* and *in what order*. The language model only *phrases*.**

This is what separates Career PathFinder from the field of LLM-wrapper chatbots that hallucinate non-existent courses and ignore prerequisites. Every recommendation is anchored to three real data sources — an authoritative skill taxonomy (O*NET-derived weights), a curated corpus of **105** real courses, and **measured** mastery from an adaptive diagnostic. The system is validated by **54 automated tests** and a Monte Carlo benchmark over **1,000** synthetic learners showing a **0.00%** prerequisite-violation rate.

2. System Architecture



The backend exposes **13 REST endpoints** plus static serving of the single-page dashboard.

---

3. Core Techniques (with honest scope)

3.1 Prerequisite Skill Graph — Kahn's Topological Sort

The skill space is a strict **Directed Acyclic Graph** of **53 nodes** and **61 prerequisite edges**. For a target role, the engine induces the relevant subgraph and orders it with **Kahn's algorithm**, using a pedagogical tie-break `(difficulty, −importance, id)` so that, among equally-unblocked skills, easier and more important ones come first. This mathematically guarantees no skill is ever scheduled before its prerequisites — verified at **0.00% violations** across 1,000 simulated learners.

3.2 Cognitive Mastery — Bayesian Knowledge Tracing (adaptive)

Mastery is **measured**, not self-reported. Each diagnostic answer updates the hidden mastery posterior via the **Corbett–Anderson BKT** model with difficulty-calibrated slip/guess/transit priors. The diagnostic is genuinely **adaptive**: it selects the next item at a difficulty near the learner's current standing, and **stops early** once mastery is confidently high (`≥ 0.85`) or low (`≤ 0.15`) — asking between 2 and 4 questions per skill instead of a fixed set.

3.3 Labour-Market Skill Gap — O*NET weighting

Required skills carry importance weights in `[0, 1]` derived from the ***ONET*** *occupational database for the six first-class roles. The gap engine ranks remaining work by* `Priority = Importance × (1 − Mastery)`*, so high-stakes deficits surface first. For arbitrary open-world roles, weights are* ***estimated heuristically*** *and clearly labelled* `"Estimated (heuristic)"` *— never presented as O*NET data.

3.4 Semantic Course Retrieval — dense embeddings + TF-IDF fallback

Course matching uses **dense semantic embeddings** (`all-MiniLM-L6-v2`), precomputed offline for both the 105-course corpus and all 53 skills. At query time the engine computes cosine similarity between the skill's vector and every course vector, then fuses that relevance with course level-fit, rating, and format into a **Multi-Factor Recommendation Score**:

```
Score(c) = 0.45·Sim(skill, course) + 0.20·LevelFit + 0.20·Rating + 0.15·FormatFit
```

Embeddings are shipped as small `.npy` files, so the **runtime needs only numpy** (no torch, no external API). If embeddings are unavailable for a skill, the engine automatically falls back to **TF-IDF lexical similarity**. Each returned course carries a `retrieval_mode` field (`semantic_embedding` or `tfidf_lexical`) for transparency.

3.5 Heuristic Open-World Role Mapper

Any career query outside the six curated roles (e.g. "Cybersecurity Penetration Tester", "Cloud DevOps Engineer") is mapped to a plausible set of DAG skills by **keyword-to-cluster affinity matching**; prerequisite ordering is then handled by the same topological sort. This is a heuristic mapper, not a learned model.

3.6 Structured Capstone Self-Assessment Rubric

Every milestone pairs with a hands-on capstone project. The `/api/milestone/evaluate` endpoint scores a learner's **declared** deliverables, architecture notes, repository link, and checklist against a 4-point rubric. It is a **self-assessment aid** (it does not inspect the repository); its pass/fail result feeds the adaptive re-planner by updating measured mastery.

3.7 Fenced Conversational Layer

Gemini 1.5 Flash is fenced strictly to (a) parsing the natural-language goal, (b) phrasing the deterministic reasoning trace, and (c) answering learner questions grounded in the active session. It never generates the curriculum. If no `GEMINI_API_KEY` is present, deterministic fallbacks keep every endpoint fully functional.

---

4. Technology Stack

Layer	Choice
API	FastAPI + Uvicorn, Pydantic-validated
Graph	NetworkX (DAG + topological sort)
Retrieval	Precomputed embeddings (numpy) + scikit-learn TF-IDF fallback
Persistence	Dual-layer in-memory cache + SQLite (WAL) session store
LLM	Gemini 1.5 Flash (fenced; optional)
Frontend	Vanilla HTML/CSS/JS, Three.js ambient scene, interactive SVG DAG explorer
Deploy	Docker / Render (single service)

Runtime dependencies are **pinned and light** — no torch, no pandas.

---

5. Validation & Empirical Results

5.1 Test suite — 54 / 54 passing (100%)

Reproduce with `python -m pytest tests/ -v`. The suite spans five comprehensive test suites:

- **`test_core.py`** — graph construction, gap ranking, BKT bounds/monotonicity, multi-factor recommendation, full end-to-end journeys, and persona benchmarks.
- **`test_coverage.py`** — diagnostic item-bank coverage across all six roles.
- **`test_integrity.py`** — DAG acyclicity/self-loop-freeness, cross-dataset synchronization (100% item-bank and project-bank coverage), O*NET weight bounds, BKT convergence, recommendation-score invariants, API error contracts, and SQLite persistence across reloads.
- **`test_enhancements.py`** — adaptive diagnostic termination and difficulty targeting, open-world gap generation, and semantic-embedding retrieval with fallback.
- **`test_security.py`** — OWASP security headers, input sanitization / regex validation, XSS escaping, prompt-injection tag neutralizing, rate limiting, and concurrent thread-safe persistence.

```
$ python -m pytest tests/ -q
..................................................      [100%]
54 passed
```

5.2 Monte Carlo benchmark — 1,000 synthetic learners

Reproduce with `python benchmarks/run_benchmark.py`.

Metric	Result
Prerequisite-violation rate	**0.0000%** (0 of 1,000)
Path generation latency	**~0.83 ms / learner**
BKT mean steps to convergence	**~2.0 questions** (matches the adaptive early-stop)
Mean multi-factor recommendation score	**0.881 / 1.0**
Curriculum compression vs static roadmap	**15.4 hours saved / learner**

5.3 Live Full-System Integration Verification

Reproduce with `python backend/scripts/full_system_test.py`.

- **20 / 20 passing (100%)** live end-to-end HTTP integration tests verifying security headers, BKT calculations, topological path generation, rubric scoring, AI coach grounding, and markdown export.

---

6. What makes this hard to build (real engineering challenges)

- **Authoring a coherent prerequisite DAG.** 53 skills and 61 directed edges had to be hand-validated to be strictly acyclic *and* cross-synchronized so every node has diagnostic items and a capstone — enforced by integrity tests rather than trust.
- **Keeping embeddings deploy-safe.** Genuine semantic retrieval usually drags in torch. We precompute vectors offline and ship them, so the runtime stays numpy-only and the image stays small — semantic quality without the deploy cost.
- **Fencing the LLM without breaking the demo.** Every LLM call has a deterministic fallback, so the product is fully functional with no API key — the model is an enhancement, never a dependency.
- **Making "adaptive" true, not decorative.** BKT tracks mastery, but the item selection and early-stop had to be wired end-to-end so the diagnostic actually shortens when confident.

Submitted for the HCL Amplified AI Learning Path Recommender Hackathon.